

Plan: c_n arity optimization pattern — three-layer lottery ticket

ops

2026-05-13

Motivation. The 2026-05-13 Bitcoin Alpha Optuna run converged on a sparse arity support $\{c_2, c_5, w_2, w_3, w_4\}$ with c_3, c_4 dropped. Closure-bearing cycles at the *endpoints* of the arity range and walks covering the middle is a candidate “complementarity” pattern worth probing at larger n . Hard-coded `c2..c5` prevents this test. We need a general c_n extension *and* an optimization pattern that does not collapse to “higher n always wins by parameter count”.

Scope. Three-layer optimization for cycle-arity selection on the `signedkan.wip` stack.

L1 – Inner training. Replace softmax/sigmoid α_k on the arity weight vector with a **Hard-Concrete L0** gate. Differentiable, pushed toward $\{0, 1\}$, with a tunable expected-L0 budget $E[\|\alpha\|_0]$.

L2 – Outer search. Add an **ASHA / successive-halving pruner** to the Optuna driver, gated on partial-epoch AUC. Composes with `HSIKAN_OPTUNA_SKIP_EXPENSIVE_ATTENTION`.

L3 – Enumeration budget. Per-arity caps proportional to $C/H(p_n)$, where $H(p_n)$ is the empirical entropy of the per-vertex cycle-count distribution at arity n . Stops c_7 from drowning c_2 by raw count.

Goal. A configurable arity range `HSIKAN_ARITY_RANGE="2,3,4,5,6,7"` runnable on the existing Bitcoin / Slashdot / Epinions pipelines, with L1-L3 gated behind env flags defaulting OFF (current behavior preserved bit-for-bit). Acceptance: after Bitcoin Alpha 5-seed paired sweep, the learned Hard-Concrete gate identifies a sparse arity support, and ASHA cuts wall time by $\geq 30\%$ vs the same 30-trial TPE-only run at equivalent best AUC.

Affected files.

- `signedkan.wip/src/mixed_arity_signedkan.py` — α_k parameterization: add Hard-Concrete gate path; soft path retained.
- `signedkan.wip/src/run_optuna_search.py` — add `OPTUNA_PRUNER=asha|hyperband|none` env switch; default `none`.
- `signedkan.wip/src/run_final_cell.py` — thread `HSIKAN_ALPHA_K_PRIOR` and `HSIKAN_ARITY_RANGE` through to the model constructor.
- `signedkan.wip/src/cycle_cache.py` — extend `_topk_fingerprint` with `HSIKAN_EQUAL_INFO_CAPS` and `HSIKAN_ARITY_RANGE` (memory `feedback_cycle_cache_fingerprint`).
- `hymeko_graph/src/topk_cycles.rs` — accept a `caps_per_arity: Vec<usize>` alongside the existing global cap; add a `EqualInfoBudget` computer in a new module `src/cycle_budget.rs`.
- `hymeko_py/src/cycles.rs` — additive `EnumerationConfig` field (memory `feedback_no_cartesian_pyfunction_dump` — no new pyfunction).
- Tests: `signedkan.wip/tests/test_alpha_k_hard_concrete.py`, `tests/test_asha_pruner.py`, `tests/test_equal_in`, `hymeko_graph/tests/cycle_budget_test.rs`.

CORE.YAML items touched. None. All edits land in `signedkan.wip`, `hymeko_graph` (non-core), and `hymeko_py` (binding glue — additive).

Interface changes.

- New env vars (all default off / current behavior):
 - `HSIKAN_ALPHA_K_PRIOR` $\in \{\text{soft}, \text{hard_concrete}\}$.
 - `HSIKAN_ARITY_RANGE` = comma-separated list, default "2,3,4,5".
 - `HSIKAN_EQUAL_INFO_CAPS` $\in \{0,1\}$.
 - `OPTUNA_PRUNER` $\in \{\text{none}, \text{asha}, \text{hyperband}\}$.
- New Rust API: `EnumerationConfig.caps_per_arity`: `Option<Vec<usize>>`. When `None`, behaviour unchanged.
- Python: `MixedAritySignedKAN(..., alpha_prior= "soft"|"hard_concrete", arity_range=tuple[int,...])`.

Hard-Concrete contract (L1). For arity weights $\alpha \in R^{|N|}$, sample $g_k \sim \text{HardConcrete}(\log \alpha_k, \beta, \gamma, \zeta)$ with stretch parameters $(\gamma, \zeta) = (-0.1, 1.1)$ and temperature $\beta = 2/3$ (Louizos et al. 2018). Loss adds $\lambda_{L0} \sum_k \Pr[g_k > 0]$, a closed-form expression in $\log \alpha_k$. At inference, gates are deterministic $\bar{g}_k = \min(1, \max(0, \sigma(\log \alpha_k)(\zeta - \gamma) + \gamma))$.

Equal-information caps (L3). For each arity n in `HSIKAN_ARITY_RANGE`, run a cheap sampled-count probe (5% of vertices) to estimate per-vertex cycle count $c_n(v)$. Compute $H_n = -\sum_v \tilde{p}_n(v) \log \tilde{p}_n(v)$ from the normalized distribution $\tilde{p}_n(v) = c_n(v) / \sum_v c_n(v)$. Allocate $\text{cap}_n = C \cdot H_n / \sum_m H_m$ where C is the total enumeration budget (default 1 M cycles). Effect: arities with flatter distributions (signal spread across many vertices) get more budget than arities concentrated on hub vertices.

Test strategy.

- **Unit:** Hard-Concrete gate is in $[0, 1]$ a.s.; gradient flows through $\log \alpha_k$ at temperature $\beta = 2/3$; expected-L0 matches closed form within 10^{-6} on a synthetic weight vector (`test_alpha_k_hard_concrete.py`).
- **Unit:** ASHA pruner integration: mocked study, half-budget cohort pruned, surviving cohort completes (`test_asha_pruner.py`).
- **Unit:** Equal-info caps: known synthetic distributions (delta, uniform, geometric) produce expected ratios within 5% (`test_equal_info_caps.py`); Rust path tested in `cycle_budget_test.rs`.
- **Integration (smoke):** Bitcoin Alpha 1-seed, 80 epochs, `HSIKAN_ARITY_RANGE="2,3,4,5,6"` with `HSIKAN_ALPHA_K_PRIOR=hard_concrete`: must complete within performance budget below; learned gate vector must contain at least one $\bar{g}_k < 0.1$ (sparsity actually fires).
- **Performance:** median/IQR/worst over 5 iterations of enumeration wall time at $n = 6$, $n = 7$ on Bitcoin Alpha; RSS measured via `memray`.
- **Production-scale smoke (CLAUDE.md §3):** 1-seed Slashdot $c_2..c_6$ at the real $n_{\text{trials}} = 1$ before any multi-seed paired run is queued.

Performance budget.

Stage	Metric	Budget
Bitcoin Alpha 1-seed, $c_2..c_6$	wall $\leq$	90 min
Bitcoin Alpha 1-seed, $c_2..c_6$	peak RSS $\leq$	12 GB
Bitcoin Alpha 1-seed, $c_2..c_6$	peak GPU mem $\leq$	7.2 GiB
Cycle enum, c_7 Slashdot	wall $\leq$	30 min
Cycle enum, c_7 Slashdot	RSS $\leq$	8 GB
ASHA 30-trial Alpha	wall vs TPE-only	$\geq 30\%$ saved

Hard cap: 16 GB RSS (CLAUDE.md §4). If c_7 Slashdot exceeds 30 min enumeration on the trial-0 cold cache, fall back to per-vertex top- K with reduced K , do not raise caps.

Rollback. All new behaviour is gated behind env flags defaulting OFF. Rollback = revert the touched files and delete the plan directory. No data, checkpoint, or pinned-dep migration is involved.

Risk anticipation.

- *Hard-Concrete temperature annealing.* If β cools too fast, α_k collapses to a single arity early and never explores; too slow, never sparsifies. Mitigation: warmup of 20% of epochs at $\beta = 2/3$, then linear to $\beta = 1/3$. Test on Bitcoin Alpha 1-seed before any multi-seed.
- *ASHA on noisy plateau.* Alpha Optuna trial 21 vs 23 differed by 0.0001. ASHA at $r = 4$ rungs may prune the eventual best. Mitigation: minimum-rung budget set to $\geq 50\%$ of full training; promote-top-fraction set to $1/2$, not $1/3$.
- *Equal-info caps interaction with existing `max_k_cap`.* The existing per-arity `max_cycles` cap (with aspirational “push the cap into Rust” comments) is a *contract*. New caps must be the `min` of (existing `max_cycles`, equal-info `cap_n`); never exceed. Test by constructing a fixture where equal-info would request more than `max_cycles` and asserting the cap holds.
- *Enumeration cost wall at c_7 .* Per memory `project_rust_cycle_enum_2026_05_02`, c_4 on Slashdot was 55M cycles in 4min. c_7 is bounded but may require per-vertex top- K + global-ABB scorer (`project_abb_global_topk_2026_05_10`) for tractability. Plan-budget assumes per-vertex top- K active. If the smoke at c_6 exceeds 12 min wall, halt and re-budget before c_7 .
- *Numerical stability of Hard-Concrete near $g_k = 0$.* Avoid catastrophic cancellation in the gate-survival probability; use `log1p/expm1` forms. Test against a `mpmath` reference at boundary $\log \alpha_k \in \{-20, 20\}$ (CLAUDE.md §5 numerical stability).

Performance-contract preservation. Worst-case input. Slashdot at `HSIKAN_ARITY_RANGE = "2,3,4,5,6,7"` with default `max_k_cap`. This is the production-scale config the plan must survive; toy fixtures at $n = 2$ are insufficient. See CLAUDE.md §3 production-scale smoke rule.

Empty-plan-dir hygiene. If this plan is abandoned before any of L1/L2/L3 is implemented, delete `docs/plans/2026-05-13-cn-optimization-pattern/` so it does not look like in-progress work.

Sequencing. Land in this order, one phase per session (memory `feedback_one_phase_per_session`): (1) L3 equal-info caps in Rust + Python dispatch + cache-fingerprint extension; (2) L1 Hard-Concrete α_k with annealing schedule; (3) L2 ASHA pruner wiring + first Bitcoin Alpha 30-trial sweep with all three enabled. Each phase ships with its tests, a smoke at production scale, and a report under `reports/`.

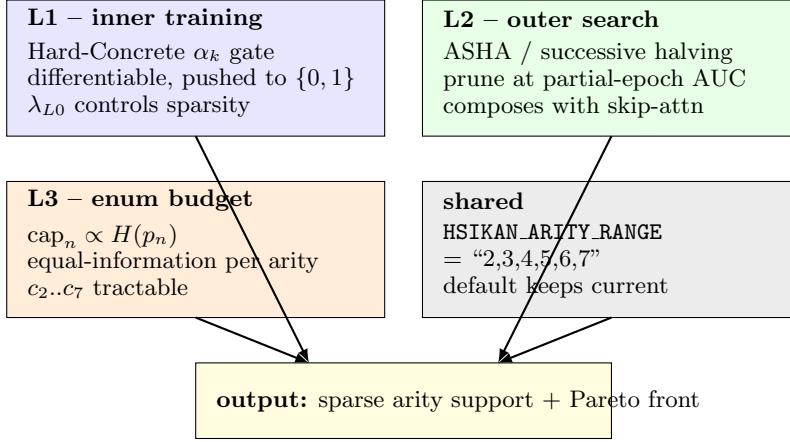


Figure 1: Three optimization layers: training-time gate (L1), outer search (L2), enumeration budget (L3). All three feed a single arity range HSIKAN_ARITY_RANGE.